



Experiment 10

Capturing & Analysing Network Packets using Wireshark

Name: Deven

UID: 24BCF10060

Aim

To capture and analyse network packets using the Wireshark packet analyzer, and to study the structure and exchange sequence of ICMP, ARP, DNS, and TCP packets.

Learning Objectives

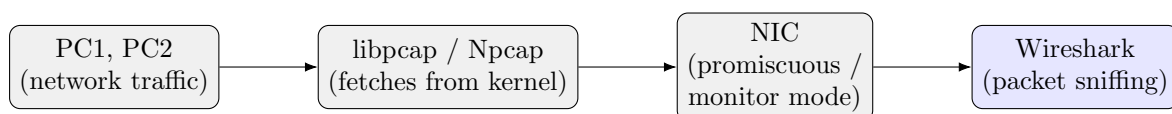
1. Understand how a packet analyzer such as Wireshark captures traffic at the network-interface level.
2. Understand promiscuous mode, monitor mode, and the role of `libpcap`/`Npcap`.
3. Choose the correct network interface before starting a capture.
4. Capture and analyse an ICMP Echo Request/Reply exchange (`ping`).
5. Capture and analyse an ARP Request/Reply exchange.
6. Capture and analyse a DNS query/response exchange (`nslookup`).
7. Capture and analyse a TCP three-way handshake and an HTTP request (`curl`).

Theory

What is Wireshark Doing Behind the Scenes?

A network interface card (NIC) normally passes a program only the packets addressed to its own device. Wireshark needs to see *all* traffic on the segment, so it puts the NIC into a special capture mode:

- **Promiscuous Mode** – on a wired/Wi-Fi connection, the NIC is told to stop discarding packets addressed to other devices, so Wireshark can capture every packet on the network segment, not just its own.
- **Monitor Mode (wireless)** – on Wi-Fi, monitor mode goes further and captures all wireless frames in the air, even ones not addressed to this device at all.
- **libpcap / Npcap** – Wireshark does not talk to the NIC directly; it relies on a capture library (`libpcap` on Linux/macOS, `Npcap` on Windows) to pull raw packets out of the OS kernel.



Protocols Commonly Captured

HTTP/HTTPS, ARP, DHCP, DNS, ICMP, and SSH/FTP are among the protocols most frequently inspected in a packet-capture lab; this experiment focuses on ICMP, ARP, DNS, and TCP.



Network Interfaces

A network interface is how a computer connects to the network – common types are **Ethernet** (wired), **Wi-Fi** (wireless), and **Loopback** (local traffic only, used for testing). Picking the wrong interface is a common beginner mistake: capturing on Loopback shows only the machine's own local traffic, and capturing on the wrong Wi-Fi adapter means the traffic being looked for is simply never seen.



Part A – ICMP Packet Capture (ping)

Procedure

1. Connect the laptop to the Internet using Ethernet, Wi-Fi, or a mobile hotspot.
2. Open **Wireshark**.
3. Select the active interface – **Ethernet** if using a LAN cable, **Wi-Fi** if using hotspot/Wi-Fi.
4. Double-click the interface to **start packet capture**.
5. Keep Wireshark capturing – do not stop it yet.
6. Open **Command Prompt (CMD)**.
7. Run **ping 8.8.8.8**.
8. Wait until the ping command finishes.
9. Return to Wireshark and click the **red Stop** button to stop capturing.
10. Apply the display filter **icmp && ip.addr == 8.8.8.8**.
11. Identify the **Echo Request** and its corresponding **Echo Reply**.

Expected Packet List

No.	Source	Destination	Len	Info
1	192.168.1.10	8.8.8.8	74	Echo (ping) request, id=0x0001, seq=1
2	8.8.8.8	192.168.1.10	74	Echo (ping) reply, id=0x0001, seq=1
3	192.168.1.10	8.8.8.8	74	Echo (ping) request, id=0x0001, seq=2
4	8.8.8.8	192.168.1.10	74	Echo (ping) reply, id=0x0001, seq=2

Protocol column (omitted above for width) reads ICMP for every row. Source IP will be the machine's own LAN address.

Packet Size Breakdown (74 bytes)

Header / Field	Size
Ethernet Header	14 bytes
IPv4 Header	20 bytes
ICMP Header	8 bytes
Ping Data	32 bytes
Total	74 bytes



Part B – ARP Packet Capture

Procedure

1. Connect the laptop to Ethernet, Wi-Fi, or a mobile hotspot.
2. Open CMD *first* and run `ipconfig`; note the **Default Gateway** IP address.
3. Open **Command Prompt as Administrator**.
4. Run `arp -d *` to clear dynamic ARP cache entries, for a clean demonstration.
5. Open **Wireshark** and select the active Ethernet/Wi-Fi interface.
6. Start packet capture and return to CMD.
7. Run `ping <Default-Gateway-IP>` – for example, `ping 192.168.0.1`.
8. After the ping finishes, return to Wireshark and click **Stop**.
9. Apply the display filter `arp`.
10. Find the **ARP Request** and its corresponding **ARP Reply**.
11. Analyse the Source, Destination, Length, Info, and the Sender/Target IP and MAC addresses.

Expected Packet List

No.	Source	Destination	Len	Info
1	Dell_lab:cd:ef	Broadcast	42	Who has 192.168.0.1? Tell 192.168.0.10
2	Netgear_11:22:33	Dell_lab:cd:ef	42	192.168.0.1 is at aa:bb:cc:11:22:33

Protocol column reads ARP for both rows. Source/Destination show the resolved vendor-prefixed MAC names Wireshark displays by default.

The ARP Request is a broadcast (`ff:ff:ff:ff:ff:ff`) asking every device on the segment "who owns this IP?"; only the device that actually owns `192.168.0.1` sends back a unicast ARP Reply containing its MAC address, which is then cached (that cache is exactly what `arp -d *` cleared beforehand).



Part C – DNS Packet Capture (nslookup)

Procedure

1. Connect the laptop to the Internet.
2. Open **Wireshark first** and select the active Ethernet/Wi-Fi interface.
3. Start packet capture and keep it running.
4. Open **CMD** and run `nslookup example.com`.
5. Wait for the DNS result to appear in CMD.
6. Return to Wireshark and click **Stop**.
7. Apply the display filter `dns.qry.name == "example.com"`.
8. Identify the **A Query** and its **Response**.
9. Analyse the Source, Destination, Protocol, Length, and Info columns.
10. Click into the packets and inspect the DNS and UDP fields.

Expected Packet List

No.	Source	Destination	Len	Info
1	192.168.1.10	192.168.1.1	74	Standard query 0x1a2b A example.com
2	192.168.1.1	192.168.1.10	90	Standard query response 0x1a2b A example.com A 93.184.216.34

Protocol column reads DNS for both rows.

DNS runs over **UDP** (typically port 53), which is why, when the DNS packet is expanded in Wireshark, a UDP header sits directly beneath the IP header – DNS does not need TCP's connection setup for a single small query/response pair.

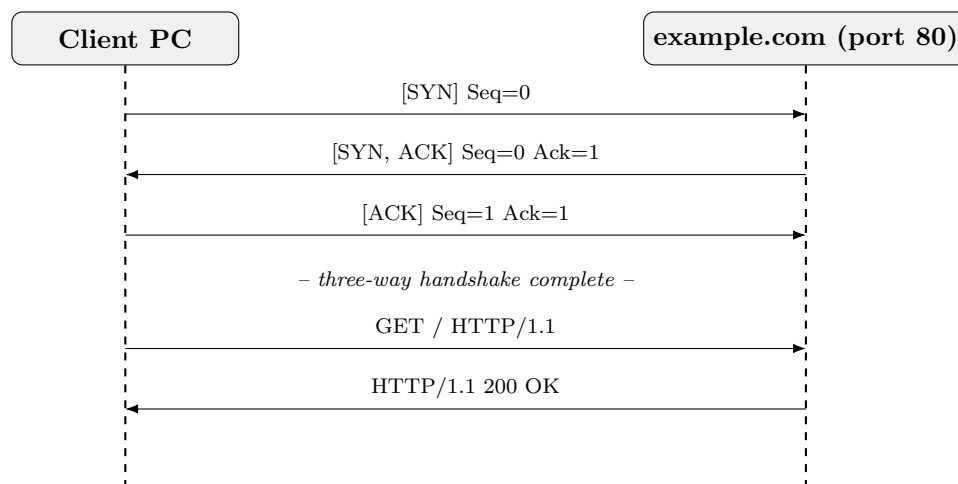


Part D – TCP Packet Capture (curl)

Procedure

1. Connect the laptop to the Internet.
2. Open **Wireshark** first and select the active Ethernet/Wi-Fi interface.
3. Start packet capture and keep it running.
4. Open **CMD** and run `curl http://example.com`.
5. Wait until the command finishes and the HTML output appears.
6. Return to Wireshark and click **Stop**.
7. Apply the display filter `tcp.port == 80`.
8. Find the TCP connection generated for `example.com`.
9. Identify `[SYN] → [SYN, ACK] → [ACK]`.
10. Examine the Source, Destination, Ports, Sequence/Acknowledgment numbers, and Flags.
11. Optionally use **Follow** → **TCP Stream** to see the HTTP GET request and the 200 OK response together.

The TCP Three-Way Handshake, as seen in the capture



Expected Packet List

No.	Protocol	Len	Info
1	TCP	66	51422 → 80 [SYN] Seq=0 Win=64240 Len=0
2	TCP	66	80 → 51422 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
3	TCP	54	51422 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
4	HTTP	148	GET / HTTP/1.1
5	HTTP	512	HTTP/1.1 200 OK (text/html)

Ports above are illustrative – the client's source port is assigned dynamically by the OS on every run.

Result

Wireshark was used to capture and analyse four different protocol exchanges on a live network interface: an ICMP Echo Request/Reply pair (confirming the standard 74-byte ping packet made up of a 14-byte Ethernet header, 20-byte IPv4 header, 8-byte ICMP header, and 32 bytes of data), an ARP Request/Reply pair resolving a gateway's MAC address, a DNS query/response pair over UDP resolving a domain name to an IP address, and a full TCP three-way handshake followed by an HTTP request/response. Together these confirm how promiscuous-mode packet capture, via `libpcap/Npcap`, exposes the layered structure of real network traffic for analysis.

Learning Outcome

After completing this experiment, the following learning outcomes were achieved.

- Understood how Wireshark uses promiscuous/monitor mode and `libpcap/Npcap` to capture packets directly from the OS kernel.
- Learned to select the correct network interface (Ethernet, Wi-Fi, or Loopback) before capturing.
- Captured and interpreted an ICMP Echo Request/Reply exchange and broke down its exact byte-level packet size.
- Captured and interpreted an ARP Request/Reply exchange and understood its role in resolving IP addresses to MAC addresses.
- Captured and interpreted a DNS query/response exchange and observed that DNS runs over UDP.
- Captured and interpreted a TCP three-way handshake and the HTTP request/response riding on top of it.
- Practised using Wireshark display filters (`icmp`, `arp`, `dns.qry.name`, `tcp.port`) to isolate traffic of interest from a full capture.